HATPro Structural Perspective

A schema-first architecture for a traveler-controlled profile

1. Purpose of This Document

This document describes the **structural organization** of the HATPro (Hospitality & Travel Profile) data model.

It is intended to answer, at a glance:

- What are the major schema components?
- How are they organized and related?
- What belongs in core vs shared libraries vs travel segments?
- How should readers mentally “navigate” the model?

This document does **not** describe:

- generator internals,
- SCHEMAHINT syntax,
- validation tooling,
- or per-field semantics.

Those details are intentionally delegated to the **Document Guide** and the underlying manuals.

---

2. HATPro at a Glance

HATPro is a **data schema project** whose primary deliverable is a **Traveler Profile JSON document**.

Key characteristics:

- Authored in **UML (PlantUML)**
- Uses embedded **Hints** to auto-generate:
    - modular JSON Schema components, and
    - JSON templates suitable for traveler (and AI-assisted) completion
- Designed to be:
    - traveler-centric,
    - segment-aware,
    - extensible without refactoring

The schema is organized as **libraries + compositions**, not a monolith.

---

3. Structural Layers (Conceptual)

From a structural perspective, the model divides cleanly into four layers:

1. **Composition Root**

2. **Core Profile Libraries**
3. **Cross-Cutting Preference & Constraint Libraries**
4. **Travel Segment Extensions**

Each layer has a distinct responsibility and dependency direction.

---

4. Composition Root

TravelProfile (top object)

```
TravelProfile
```

- The single **top-level composition object**
- Exists to *assemble* the profile, not to define domain semantics
- References (composes) all major profile areas

Design rule:
The top object should remain *stable, minimal, and boring*.

---

5. Core Profile Libraries (Who the Traveler Is)

These packages define **general traveler data**, independent of any specific trip or service.

```
Base_Profile
├─ Identity
├─ ReligionCultureInfo
├─ Lifestyle
|   └─ IdentityPrefs
├─ PersonalCommInfo
├─ CommChannels
|   └─ (email, phone, SSI, messaging, etc.)
└─ PhysicalLocation
    └─ (addresses, locations)
```

Structural intent

- Durable, self-asserted data
- Shared across all travel contexts
- Not credentials, not bookings, not transactions

This layer answers:

> "Who is this traveler, and how do they wish to be identified and contacted?"

---

6. SupportNeeds (Operational Constraints)

SupportNeeds captures **operationally relevant needs**, expressed declaratively and non-clinically.

```
SupportNeeds
├─ Medical / Medications
├─ Accessibility & Mobility
├─ TravelSensitivities
└─ SubstanceExposureRisks
    └─ (Allergies++)
```

Structural intent

- Focus on *what providers need to know to act*
- Avoid diagnoses or medical records
- Support graded, explainable constraints

This layer answers:

> "What must be respected or accommodated to safely and effectively serve this traveler?"

---

7. Preferences (How the Traveler Likes Things)

Preferences are modeled as a **reusable, structured system**, not ad-hoc enums.

7.1 Preference Infrastructure

```
Preferences
├─ lib
|   ├─ preference_constraint_model
|   └─ node_edge_acyclic_graph
```

- Preferences are nodes in a **directed acyclic graph**
- Explicit edge semantics support inheritance, constraints, and overrides
- This library is reused everywhere preferences appear

7.2 Common Preference Domains

```
Preferences
└─ Common
   ├─ Unclassified (General)
   ├─ Bedding
   ├─ Seating
   └─ FoodAndDining
      ├─ FoodAndBeveragePrefs
      ├─ DiningExperiencePrefs
      ├─ Diets
      └─ CuisinePrefs
```

Structural intent

- These domains apply across **multiple travel segments**
- They live once, centrally
- Segments *reference* them rather than redefining them

This layer answers:

> "What does the traveler generally prefer, regardless of context?"

---

8. Travel Segments (Context-Specific Extensions)

Segments represent **contextual specializations** of the profile.

```
Segments
├─ AirTravel
├─ Train
├─ Ship_Cruise
├─ Lodging
|   └─ (room features, floor, nearTo, etc.)
├─ CarRental
├─ RVRental
└─ Experiences
   ├─ Skiing
   ├─ Scuba
   └─ Entertainment
```

Structural intent

- Segment packages:
  - reuse Base_Profile, SupportNeeds, and Preferences
  - add only what is meaningful *in that context*
- No segment depends on another segment
- New segments can be added without destabilizing the core

This layer answers:

> "What additional data matters *only* in this travel context?"

---

9. Integrated Structural Graph (Text View)

Below is a **simplified structural graph**, read top-down:

```
TravelProfile
|
├─ Base_Profile
|   ├─ Identity
|   ├─ ReligionCultureInfo
|   ├─ Lifestyle
|   |   └─ IdentityPrefs
|   ├─ PersonalCommInfo
|   ├─ CommChannels
|   └─ PhysicalLocation
|
├─ SupportNeeds
|   ├─ Medical / Medications
|   ├─ Accessibility & Mobility
|   ├─ TravelSensitivities
|   └─ SubstanceExposureRisks (Allergies++)
|
├─ Preferences
|   ├─ lib
|   |   ├─ preference_constraint_model
|   |   └─ node_edge_acyclic_graph
|   └─ Common
|       ├─ Bedding
|       ├─ Seating
|       └─ FoodAndDining
|           ├─ FoodAndBeveragePrefs
|           ├─ DiningExperiencePrefs
|           ├─ Diets
|           └─ CuisinePrefs
|
└─ Segments
    ├─ AirTravel
    ├─ Train
    ├─ Ship_Cruise
    ├─ Lodging
    ├─ CarRental
    ├─ RVRental
    └─ Experiences
        ├─ Skiing
        ├─ Scuba
        └─ Entertainment
```

How to read this graph

- Vertical = **composition**
- Horizontal siblings = **peer packages**
- Lower levels refine or specialize higher ones
- No arrows point upward: dependencies flow **downward and inward**

---

10. Why This Structure Matters

This structure enables:

- **Schema stability** over time
- **AI-assisted profile completion** using generated templates
- **Purpose-limited data sharing** (core vs segment data)
- **Standards-friendly modularization**
- **Incremental evolution** without refactoring existing profiles

Most importantly:

> The model scales by *addition*, not by restructuring.

---

11. Relationship to Other Documentation

- This document explains **structure and intent**
- The **Document Guide** explains *where to find details*
- The developer manuals explain *how to implement, generate, and validate*

Together, they form a layered documentation set:

- **Structural perspective** (this document)
- **Navigational perspective** (Document Guide)
- **Procedural detail** (manuals)